

The objective of this lab is to:

1. Understand and practice pointer to object and array of objects.
2. Practice good coding conventions e.g commenting, meaningful variable and functions names, properly indented and modular code.

Instructions!

1. This is a **graded** lab, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed asking how is he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
2. Indent your code properly.
3. Use meaningful variable and function names. Use the camelCase notation.
4. Save your work frequently. Make a habit of pressing **CTRL+S** after every line of code you write.
5. **Do NOT use any global or static variable(s). However, global named constants may be used.**
6. **Make sure that there are NO dangling pointers or memory leaks in your programs.**
7. **Make sure that all functions have been implemented and can be called in main function. Otherwise, no marks will be awarded for that function.**

Task 01:**[40 Marks]**

Create a C++ class. The name of the class that you create should be **Student**. Use the following data types for member variables of the structure **Student**:

Name	string
ID Number	int
Tests	int*
Average	double
Grade	char

Apart from the main function, your implementation **MUST** have the following functions in Student class:

1. Student* allocateArray (int&, int&);

This function should take two inputs from the user and store these into the two reference parameters. After that this function should dynamically allocate the array of **Students** based on the user's input. First parameter is the size of array of Students object while the second parameter is the size of tests array. At the end, this function will return a pointer to the allocated array.

2. void getInput (Student* arr, int numStudents, int numTests);

This function will be used to take the input from the user. This function should ask the user for each Student's *Name*, *ID Number*, and his/her *Test scores*. The *Average test score* and *Grade* will NOT be taken from the user. Rather, these will be calculated later on.

3. void calcAvgAndGrade (Student* arr, int numStudents, int numTests);

This function will be used to calculate and store the *Average test score* and *Grade* for all students. No input or output should be performed in this function.

Moreover, you should call this function inside function 2.

While calculating grade, use average score you have just calculated for grading:

Average Score	Grade
Greater or Equal to 90	A
Greater or Equal to 80 but less than 90	B
Greater or Equal to 70 but less than 60	C
Greater or Equal to 60 but less than 50	D
Greater or Equal to 50 but less than 60	E
Less than 50	F

4. void displayOneStudent (const Student& st, int numTests);

This function will display all details of the student object **st** on screen. When implementing this function, you will assume that the Average test score and Grade has already been computed for the student object **st**.

5. void displayStudents (Student* arr, int numStudents, int numTests);

This function will be used to display the details of all students of screen. In the implementation of this function, you **MUST** call the **displayOneStudent(...)** function, for each Student object, which you have implemented above in **Task 4**. When implementing this function, you will assume that the Average test score and Grade has already been computed for all students in the array **arr**.

6. int getStudentCount (Student* arr, int numStudents, char grade);

This function will be used to calculate and return the count of the students that have obtained a particular **grade**. For example, if the last parameter is **'A'**, then this function should return the count of the students that obtained an **'A'** grade in the course. No input or output should be performed in this function.

7. Student* getBestStudent (Student* arr, int numStudents);

This function will search the array of Students and determine the Student which has obtained the **maximum average score**. This function should return a pointer to the Student object which has obtained the maximum average score. No input or output should be performed in this function. If more than one student has secured the maximum average score, then this function should return a pointer the Student which occurs first in the array **arr**.

8. void displayBestStudent (Student* arr, int numStudents, int numTests);

This function will display all the details of the Student which has obtained the maximum average score. In this function, you **MUST** use the functions from **Task 7** and **Task 4**.

9. void deallocateArray (Student*& arr, int numStudents, int numTests);

This function will be used to deallocate the dynamically allocated array of Students **arr**. Make sure that your code does not generate memory leaks or dangling pointers.

Note: You are NOT allowed to modify the prototype of any of the above functions. Also, make sure that you follow all instructions mentioned in the description of each of the above functions.

The flow of your program should be:

- i.* Take data of all students from the user.
- ii.* Calculate the average test scores and grades for all students.
- iii.* Display the data of all students on screen in a neat and readable way.
- iv.* Display the data of the best student (having maximum average score) on screen
- v.* Display the count of students which obtained grade A; then display count of student which obtained grade B; and similarly for grades C, D, and F.
- vi.* Finally, deallocate all the dynamically allocated memory.